

LABORATÓRIO TPSE I



Lab 08: Desenvolvendo Device Drivers

Prof. Francisco Helder

2 de fevereiro de 2025

1 Módulos no Kernel

Os Device Drivers assumem um papel especial no kernel do Linux. São “caixas pretas” distintas que fazem uma determinada peça de hardware responder a uma interface de programação interna bem definida; eles escondem completamente os detalhes de como o dispositivo funciona. As atividades do usuário são realizadas por meio de um conjunto de chamadas padronizadas e independentes do driver específico; mapear essas chamadas para operações específicas do dispositivo que atuam no hardware real é, então, a função do Device Driver.

Esta interface de programação é tal que os drivers podem ser construídos separadamente do resto do kernel e “plugados” em tempo de execução quando necessário. Essa modularidade torna os drivers em Linux fáceis de escrever, a ponto de agora existirem centenas deles disponíveis.

Conforme você aprende a escrever drivers, você descobre muito sobre o kernel Linux em geral; isso pode ajudá-lo a entender como sua máquina funciona e por que as coisas nem sempre são tão rápidas quanto você espera ou não fazem exatamente o que você deseja.

2 Carregando Módulos no Linux

Um dos bons recursos do Linux é a capacidade de estender em tempo de execução o conjunto de recursos oferecidos pelo kernel. Isso significa que você pode adicionar funcionalidades ao kernel (e remover funcionalidades também) enquanto o sistema está funcionando.

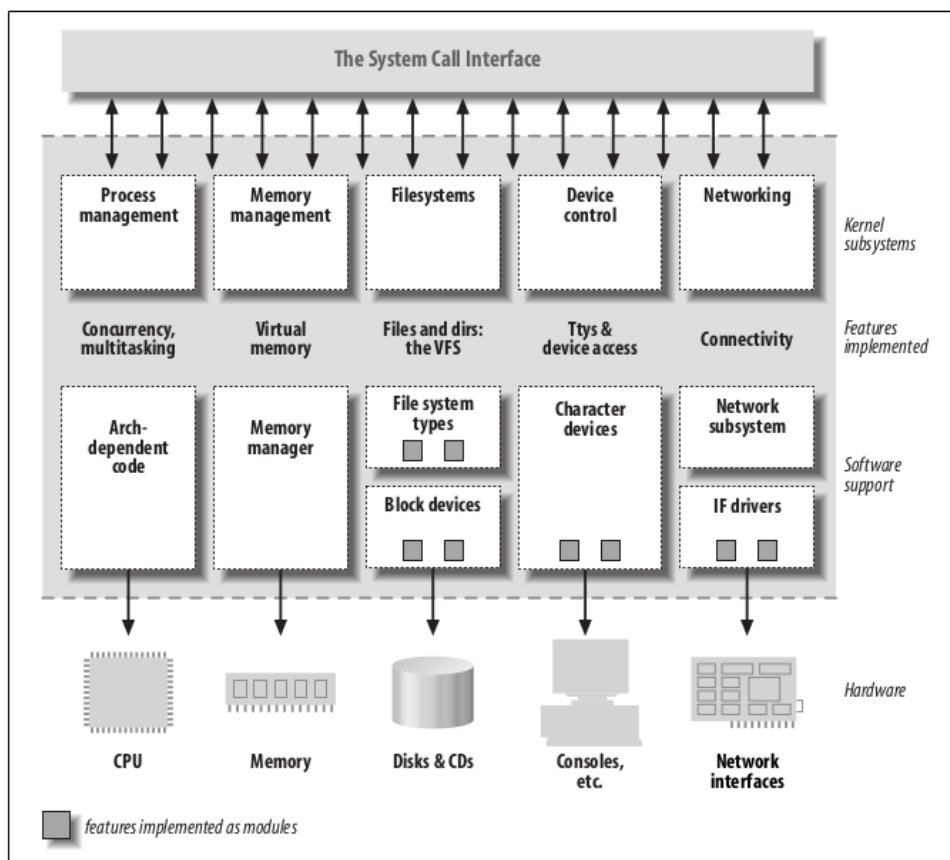


Figura 1: Um diagrama de blocos do Kernel.

Cada pedaço de código que pode ser adicionado ao kernel em tempo de execução é chamado de módulo. O kernel do Linux oferece suporte para vários tipos (ou classes) de módulos diferentes, incluindo, mas não limitado a, drivers de dispositivo. Cada módulo é composto de código objeto (não vinculado a um executável completo) que pode ser vinculado dinamicamente ao kernel em tempo de execução digitando **insmod** e pode ser desvinculado digitando **rmmod**.

A Figura 1 identifica diferentes classes de módulos responsáveis por tarefas específicas – diz-se que um módulo pertence a uma classe específica de acordo com a funcionalidade que oferece. A colocação dos módulos na Figura 1 cobre as classes mais importantes, mas está longe de ser completa porque cada vez mais funcionalidades no Linux estão sendo modularizadas.

3 Classes de Devices e Módulos

A forma como o Linux vê os dispositivos distingue entre três tipos fundamentais de dispositivos. Cada módulo geralmente implementa um desses tipos e, portanto, é classificável como **char module**, **block module** ou **network module**. Essa divisão de módulos em diferentes tipos, ou classes, não é rígida; o programador pode optar por construir módulos enormes implementando diferentes drivers em um único pedaço de código. Essas três classes são:

Character devices: Um dispositivo de caractere (char) é aquele que pode ser acessado como um fluxo de bytes (como um arquivo); um driver char é responsável por implementar esse comportamento. Esse driver geralmente implementa pelo menos as chamadas de sistema, tais como **open**, **close**, **read** e **write**. O console de texto (**/dev/console**) e as portas seriais (**/dev/ttyS0**) são exemplos de dispositivos char, pois são bem representados pela abstração de fluxo. Os dispositivos Char são acessados via sistema de arquivos, tais como **/dev/ttyUSB0** e **/dev/lp0**.

Block devices: Assim como os dispositivos char, os dispositivos de bloco são acessados via sistema de arquivos no diretório **/dev**. Um dispositivo de bloco é um dispositivo (por exemplo, um disco) que pode hospedar um sistema de arquivos. Na maioria dos sistemas Unix, um dispositivo de bloco só pode lidar com operações de E/S que transferem um ou mais blocos inteiros, que geralmente têm 512 bytes (ou uma potência maior de dois) bytes de comprimento.

Network interfaces: Qualquer transação de rede é feita por meio de uma interface, ou seja, um dispositivo capaz de trocar dados com outros hosts. Normalmente, uma interface é um dispositivo de hardware, mas também pode ser um dispositivo de software puro, como a interface de loopback. Uma interface de rede é responsável por enviar e receber pacotes de dados, acionados pelo subsistema de rede do kernel, sem saber como as transações individuais são mapeadas para os pacotes reais que estão sendo transmitidos.

Existem outras maneiras de classificar os módulos de driver que são ortogonais aos tipos de dispositivos acima. Em geral, alguns tipos de drivers funcionam com camadas adicionais de funções de suporte ao kernel para um determinado tipo de dispositivo. Por exemplo, pode-se falar de módulos de barramento serial universal (USB), módulos seriais, módulos SCSI e assim por diante. Cada dispositivo USB é acionado por um módulo USB que funciona com o subsistema USB, mas o próprio dispositivo aparece no sistema como um char device (uma porta serial USB, digamos), um dispositivo de bloco (um leitor de cartão de memória USB) ou um dispositivo de rede (uma interface Ethernet USB).

4 Configurando, Compilando e Instalando os Módulos

Atenção: Gerar a imagem do kernel e o sistema de arquivo conforme realizado nas práticas 03 e 04.

No terminal do Linux entre no diretório do código-fonte do kernel ou do ambiente do buildroot e então entre no menu de configurações do kernel, para realizar alterações para o módulo de `usb_storage`, ou seja para poder conectar um pendrive no usb da placa:

```
$ cd ~/XXX/lab_03/linux
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- menuconfig
```

Em seguida acesse o menu “Device Drivers” e o submenu “SCSI device support” e marque com um “M”, para o kernel inicialize esse dispositivo no modo módulo e não no modo in-built, como visto da Figura 2.

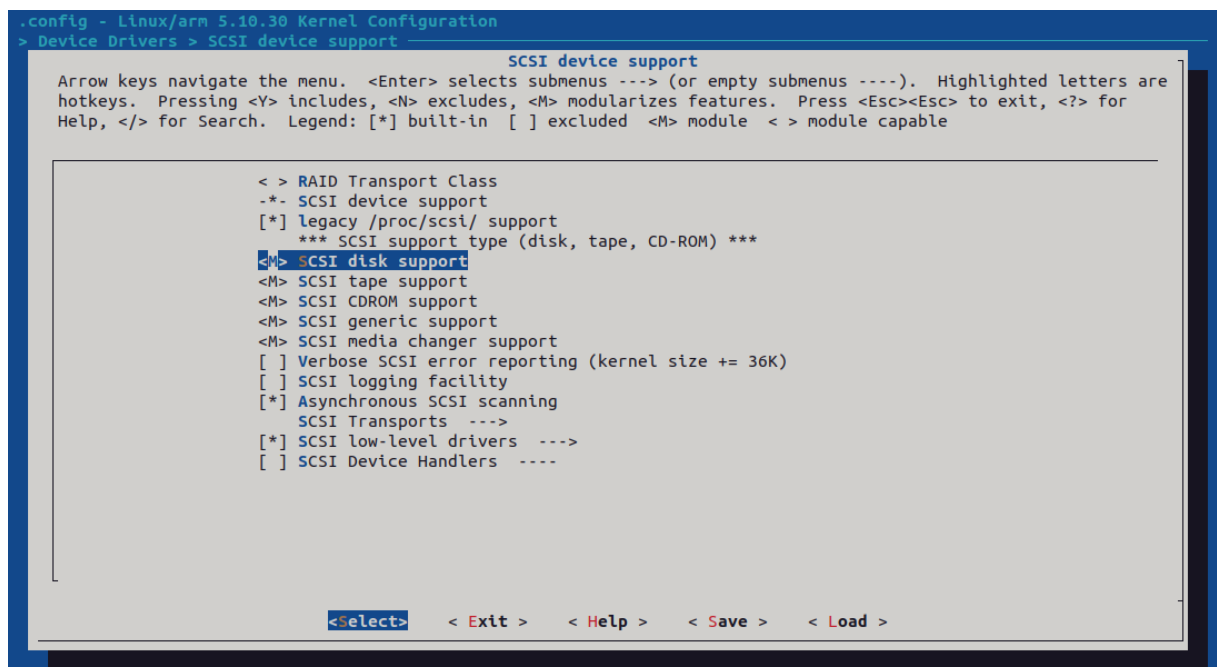


Figura 2: Processos rodando no linux gerado.

Ainda no menu “Device Drivers”, acesse o submenu “USB support” e marque com um “M”, o suporte a dispositivos de armazenamento USB, opção “USB Mass Storage support” como mostrado na Figura 3.

Salve e saia do menu de configuração. Agora o kernel está pronto para ser compilado, então execute o seguinte comando::

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- modules -j4
```

E agora instale os módulos no rootfs.

```
$ make ARCH=arm INSTALL_MOD_PATH=/XXX/rootfs modules_install
```

Perceba que o kernel instalou os módulos em `lib/modules/`

```
$ ls nfs/lib/modules/5.10.131/kernel/drivers/usb/storage/
usb-storage.ko
```

Inicialize a placa Insira um pendrive na porta usb da Beaglebone, perceba que o linux não conseguiu identificar o pendrive. Então insira o modulo com os seguintes comandos:

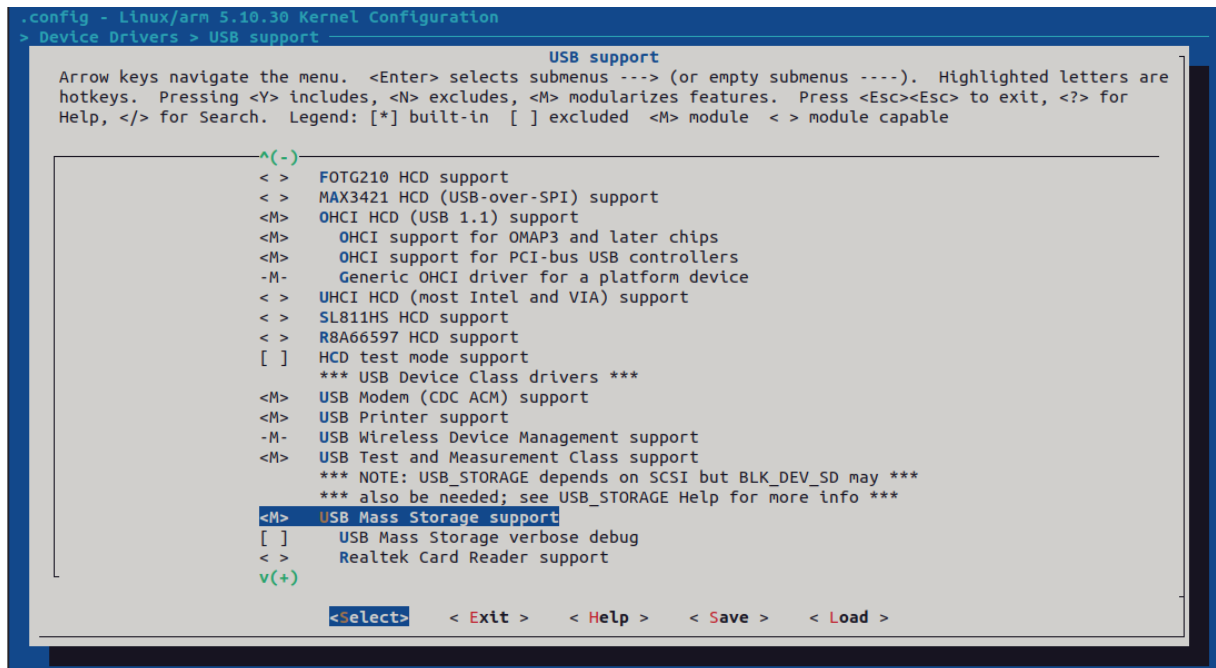


Figura 3: Processos rodando no linux gerado.

```
$ insmod /lib/modules/4.1.30/kernel/drivers/usb/storage/usb-storage.ko
$ modprobe usb-storage
```

Agora o Linux iniciou o modulo usb de armazenamento e está pronto para ser utilizando, como mostra a Figura 4

```
#
[ 31.165423] usb 1-1: new high-speed USB device number 2 using musb-hdrc
[ 31.346475] usb 1-1: New USB device found, idVendor=0781, idProduct=5567, bcdDevice= 1.26
[ 31.354748] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 31.362069] usb 1-1: Product: Cruzer Blade
[ 31.366288] usb 1-1: Manufacturer: SanDisk
[ 31.370432] usb 1-1: SerialNumber: 200608770007B1829FF7
[ 31.388126] PM: Cannot get wkup_m3_ipc handle
[ 31.473801] usb-storage 1-1:1.0: USB Mass Storage device detected
[ 31.481907] scsi host0: usb-storage 1-1:1.0
[ 31.486953] usbcore: registered new interface driver usb-storage
[ 31.493341] PM: Cannot get wkup_m3_ipc handle
[ 32.567805] scsi 0:0:0:0: Direct-Access SanDisk Cruzer Blade 1.26 PQ: 0 ANSI: 5
[ 32.658681] sd 0:0:0:0: [sda] 7821312 512-byte logical blocks: (4.00 GB/3.73 GiB)
[ 32.668055] sd 0:0:0:0: [sda] Write Protect is off
[ 32.673638] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[ 32.690947] sda:
[ 32.698787] sd 0:0:0:0: [sda] Attached SCSI removable disk
[ 32.708237] PM: Cannot get wkup_m3_ipc handle
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.8 | VT102 | Offline | ttyUSB0
```

Figura 4: Processos rodando no linux gerado.

5 Escrevendo um Módulo

Nesta seção, vamos escrever e executar um módulo completo (embora relativamente sem utilidade) e examinar alguns dos códigos básicos compartilhados por todos os módulos. De-

envolver esse conhecimento é uma base essencial para qualquer tipo de driver modular. Para evitar lançar muitos conceitos de uma só vez, vamos focar apenas sobre módulos, sem se referir a nenhuma classe de dispositivo específica.

5.1 Configurando seu Módulo de Teste

O módulo de exemplo devem funcionar com qualquer kernel 5.10.x, incluindo aqueles fornecidos por fornecedores de distribuição. No entanto, recomendamos que você utilize o kernel que você obteve na prática 03.

5.2 O Módulo Hello

Todas as aulas de programação começam com um exemplo “hello world” como forma de mostrar o programa mais simples possível. Esta prática trata de módulos do kernel em vez de programas; então, o código a seguir é um módulo “hello world” completo. Nesta prática iremos criar o driver seguindo os seguintes passos:

1. Crie um diretório para o driver compilado:

```
$ mkdir /XXX/rootfs/lib/modules/5.10.XX/kernel/drivers/hello
```

2. Crie um diretório para o código fonte:

```
$ mkdir /XXX/driver_hello/  
$ cd driver_hello
```

3. Crie o arquivo **hello.c** em **/driver_hello**, com o seguinte conteúdo:

```
1 #include <linux/module.h>  
2 #include <linux/kernel.h>  
3  
4 #error Are we building this file?  
5  
6 static int hello_init(void){  
7     printk(KERN_ALERT ``Hello, world\n'');  
8     return(0);  
9 }  
10  
11 static void hello_exit(void){  
12     printk(KERN_ALERT ``Goodbye, cruel world\n'');  
13 }  
14  
15 module_init(hello_init);  
16 module_exit(hello_exit);  
17 MODULE_LICENSE("GPL");  
18 MODULE_AUTHOR("Author_name<author@email.com>");  
19 MODULE_DESCRIPTION("Test_kernel_module");  
20 MODULE_VERSION("1.0");
```

4. Altere as informações do módulo para incluir seus dados.

5. Após o **#include** é adicionado a seguinte linha: **#error Are we building this file?**

- Quando compilado, isso gerará um erro que provará que você está construindo este arquivo.
- Mais tarde, quando tiver certeza de que seu arquivo está sendo compilado corretamente, você comenta essa linha, mas no momento ela servirá como nosso teste de que o processo está funcionando.

6. Crie Makefile em `/XXX/driver_hello/` com o seguinte conteúdo:

```

1 # Makefile for driver
2 # if KERNELRELEASE is defined, we've been invoked from the
3 # kernel build system and can use its language.
4 ifneq (${KERNELRELEASE},)
5     obj-m := hello.o
6     # Otherwise we were called directly from the command line.
7     # Invoke the kernel build system.
8 else
9     KERNEL_SOURCE := ${HOME}/XXX/lab_03/linux
10    PWD := $(shell pwd)
11    # Linux kernel 5.10 (one line)
12    CC=${HOME}/XXX/toolchain/arm-linux-gnueabi/bin/arm-linux-gnueabi-
13    BUILD=boneblack
14    CORES=4
15    image=zImage
16    PUBLIC_DRIVER_PWD=${HOME}/XXX/roots/lib/modules/5.10.131/kernel/drivers/hello
17 default:
18     # Trigger kernel build for this module
19     $(MAKE) -C ${KERNEL_SOURCE} M=${PWD} -j${CORES} ARCH=arm \
20     LOCALVERSION=${BUILD} CROSS_COMPILE=${CC} ${address} \
21     $(image) modules
22     # copy result to public folder
23     cp *.ko ${PUBLIC_DRIVER_PWD}
24 clean:
25     $(MAKE) -C ${KERNEL_SOURCE} M=${PWD} clean
26 endif

```

• Importante:

- O nome do arquivo deve ser Makefile (o kernel é case sensitive!); não o nomeie como “makefile”.
- A instrução “CC” é uma linha, sem espaços ou avanços de linha.
- Leia os comentários para entender o que está acontecendo. Os básicos são:
 - Caso a seguinte afirmação seja falsa: **ifneq** (\${KERNELRELEASE},) então o **make** irá executar a parte “else” da declaração “ifneq”. Isso configura parâmetros para o sistema de compilação do kernel e, em seguida, chama o sistema de compilação do kernel solicitando que ele compile essa pasta.
 - A compilação do kernel começa a ser executada (via **make**) na pasta deste device driver.
 - Caso a seguinte afirmação seja verdadeira: **ifneq** (\${KERNELRELEASE},) então ele adiciona nosso(s) driver(s) à variável obj-m, que é usada pelo resto do sistema de construção do kernel para construir drivers.
 - Observe que este Makefile sozinho não é suficiente para construir seu driver por conta própria; ele aproveita o sistema geral de compilação do kernel.
- 7. Teste se seu Makefile funciona para construir seu código fazendo com que a diretiva **#error** quebre a build:

```

driver_hello$ make
# Trigger kernel build for this module
make -C /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux M=/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello
LOCALVERSION=boneblack CROSS_COMPILE=/home/helderccs/UFC/disciplinas/QXD0150/lab/toolchain/arm-linux-gnueabi/bin/arm-linux-gnueabi-
\
zImage modules
make[1]: Entering directory '/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux'
Kernel: arch/arm/boot/Image is ready
CC [M] /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.o
/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.c:7:2: error: #error Are we building this file?
#error Are we building this file?
~~~~~
make[2]: *** [scripts/Makefile.build:280: /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.o] Error 1
make[1]: *** [Makefile:1825: /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello] Error 2
make[1]: Leaving directory '/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux'
make: *** [Makefile:19: default] Error 2

```

Você deve ver a mensagem **error: #error Are we building this file?.** Se sim, ótimo! Você está construindo o arquivo que espera! Comente a instrução **#error** em seu `hello.c` e continue; caso contrário, verifique se o `Makefile` e a pasta de compilação do kernel estão corretos.

8. Construa o driver somente rodando **make**, quando tiver sucesso, ficará assim:

```
# Trigger kernel build for this module
make -C /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux M=/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello
LOCALVERSION=boneblack CROSS_COMPILE=/home/helderccs/UFC/disciplinas/QXD0150/lab/toolchain/arm-linux-gnueabi/bin/arm-linux-gnu
\
zImage modules
make[1]: Entering directory '/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux'
Kernel: arch/arm/boot/Image is ready
CC [M] /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.o
MODPOST /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/Module.symvers
CC [M] /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.mod.o
Kernel: arch/arm/boot/zImage is ready
LD [M] /home/helderccs/UFC/disciplinas/QXD0150/lab/lab_08/driver_hello/hello.ko
make[1]: Leaving directory '/home/helderccs/UFC/disciplinas/QXD0150/lab/lab_03/linux'
# copy result to public folder
cp *.ko /home/helderccs/UFC/disciplinas/QXD0150/lab/nfs/lib/modules/5.10.131/kernel/drivers/hello
```

- Confira que o arquivo `.ko` foi corretamente copiado para o diretório `/XXX/roofs/lib/modules/5.10.XX/kernel/drivers/hello`

```
$ ls nfs/lib/modules/5.10.XX/kernel/drivers/hello/
hello.ko
```

5.3 Manipulando o Driver

Para usar o driver, ele deve estar disponível na placa Beaglebone em tempo de execução. No destino, mude para o diretório que contém o arquivo `.ko`. Se montado via NFS:

```
$ cd lib/modules/5.10.XX/kernel/drivers/hello/
```

Liste os módulos carregados na placa, e você verá que o `helloDriver` não está listado:

```
$ lsmod
```

Carregue o driver:

```
$ insmod hello.ko
```

Você pode não ver nenhuma saída na tela, pois o driver imprime apenas no log do kernel. Para ver isso, talvez seja necessário executar **dmesg**:

```
$ dmesg | tail -1
[ 938.788651] Hello, wolrd
```

Se você vir **insmod: cannot insert 'testdriver.ko': invalid module format**, provavelmente significa que o kernel atual foi compilado com uma string de versão diferente da que seu host está compilando atualmente. Reconstrua e baixe o kernel ou altere o host para compilar a mesma versão que o destino (encontrado executando `uname -r`). Caso você vir uma mensagem **loading out-of-tree module taints kernel**, você pode ignorá-la.

Veja o módulo carregado na placa, então você deve ver o `hello` carregado.

```
$ lsmod
```

Para remover o driver na placa, você deve executar o seguinte comando:

```
$ rmmod hello
Goodbye, cruel world
```


5.4 Passando Parâmetros para Módulo

Os módulos podem receber argumentos de linha de comando, mas não com o `argc/argv` como você está acostumado. Para permitir que argumentos sejam passados para o seu módulo, declare as variáveis que assumirão os valores dos argumentos da linha de comando como globais e então use a macro `module_param()`, (definida em `linux/moduleparam.h`) para configurar o mecanismo. Em tempo de execução, o `insmod` preencherá as variáveis com quaisquer argumentos de linha de comando fornecidos, como `./insmod mymodule.ko myvariable=5`. As declarações de variáveis e macros devem ser colocadas no início do módulo para maior clareza.

A macro `module_param()` leva 3 argumentos: o nome da variável, seu tipo e permissões para o arquivo correspondente em `sysfs`. Os tipos inteiros podem ser assinados normalmente ou não assinados. Se você quiser usar matrizes de números inteiros ou strings, consulte `module_param_array()` e `module_param_string()`, como visto no exemplo abaixo.

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3
4 MODULE_LICENSE("GPL");
5
6 static char *whom = "world";
7 module_param(whom, charp, 0644);
8 MODULE_PARM_DESC(whom, "Recipient_of_the_hello_message");
9
10 static int howmany = 1;
11 module_param(howmany, int, 0644);
12 MODULE_PARM_DESC(howmany, "Number_of_greetings");
13
14 static int __init hello_init(void) {
15     int i;
16     for (i = 0; i < howmany; i++)
17         pr_alert("(%d)_Hello,_%s\n", i, whom);
18     return 0;
19 }
20
21 static void __exit hello_exit(void) {
22     pr_alert("Goodbye,_%s\n", whom);
23 }
24 module_init(hello_init);
25 module_exit(hello_exit);
```

6 Descrevendo Device de Hardware

6.1 Criando um Device Tree Personalizada

Para permitir que o kernel Linux lide com um novo dispositivo, precisamos adicionar uma descrição deste dispositivo no Device Tree da placa. Como o Device Tree da Beaglebone Black é fornecida pela comunidade do kernel e continuará a evoluir por conta própria, não queremos fazer alterações diretamente no arquivo do DT desta placa.

A maneira mais fácil de personalizar a placa DTS é criar um novo arquivo DTS que inclua o Beaglebone Black ou Black Wireless DTS e adicionar suas próprias definições. Portanto, crie um novo arquivo `am335x-boneblack-custom.dts` no qual você inclui apenas o arquivo DTS normal da placa.

```
// SPDX-License-Identifier: GPL-2.0
#include "am335x-boneblack-wireless.dts"
```

Modifique o arquivo [arch/arm/boot/dts/Makefile](#) para adicionar seu Device Tree personalizada e, em seguida, compile-a com (`make dtbs`). Agora, copie o novo DTB para o diretório inicial do servidor tftp, altere o nome do arquivo DTB na configuração do U-Boot e inicialize a placa.

6.2 Configurando o Nome do Modelo da Placa

Modifique o arquivo do Device Tree personalizado para substituir o nome do modelo do seu sistema. Defina a propriedade do modelo como **TPSE BeagleBone Black**. Recompile o Device Tree e reinicie a placa com ela. Você deverá ver o novo nome do modelo em dois lugares diferentes:

- Nas primeiras mensagens do kernel no console serial.
- Em `/sys/firmware/devicetree/base/model`. Isso pode ser útil para uma distribuição identificar o dispositivo em que está sendo executada.

6.3 Definindo LEDs

O BBB possui quatro LEDs de usuário (USR0, USR1, USR2, USR3) no canto próximo ao conector micro-USB. Quando a placa está em execução, nenhum deles está habilitado no momento. Comece examinando os diferentes arquivos de descrição e procure um nó que definiria os LEDs.

Na verdade, os quatro LEDs devem ser acionados por um driver que corresponda aos leds gpio compatíveis. Este é um driver genérico que atua em LEDs conectados a GPIOs. Mas como você pode observar, apesar de fazerem parte do Device Tree em uso, os LEDs permanecem apagados. A razão para isso é a ausência de driver para este nó: nada realmente aciona os LEDs, mesmo que sejam descritos. Portanto, você pode começar recompilando seu kernel com `CONFIG_LEDS_CLASS=y` e `CONFIG_LEDS_GPIO=y`.

Agora você deve ver um dos LEDs piscando com a atividade da CPU, os outros permanecem apagados. Se você olhar os documentos de vinculação [Documentation/devicetree/bindings/leds/common.yaml](#) e [Documentation/devicetree/bindings/leds/leds-gpio.yaml](#), notará que podemos ajustar o estado padrão para tornar os três LEDs de usuário inativo.

Você precisará modificar um arquivo DTSI compartilhado para fazer isso. Mas como não queremos impactar outras placas que também usam o mesmo arquivo DTSI, podemos, em vez disso, adicionar um rótulo ao nó do contêiner leds. Poderíamos então referenciar esse novo rótulo em nosso DTS personalizado e substituir a propriedade de estado padrão de cada subnó de LED.

Reinicie a placa usando o novo DTS e observe a mudança de estado padrão dos LEDs, exceto um deles que não acenderá. É esperado. Se você olhar novamente para o arquivo comum que define os LEDs, eles estão na verdade todos vinculados a um **linux,default-trigger**. O estado padrão só se aplica até que o gatilho inicie sua atividade. Aquele que permanece desligado é na verdade acionado pelo gatilho eMMC, você pode tentar fazê-lo piscar criando um pouco de atividade no barramento eMMC (mas não sobrescreva seu conteúdo!).

Um dos LEDs restantes é acionado pelo cartão SD e o outro deve ser uma pulsação, que você pode ativar com `CONFIG_LEDS_TRIGGER_HEARTBEAT=y`.

7 Atividades Práticas

pratica 1

Crie, compile e carregue seu primeiro módulo e adicione parâmetros ao módulo.

pratica 2

Navegue e atualize árvores Device Tree, manipulando os GPIOs para LEDs e configura o PWM no barramento expensor.